

Procesadores Segmentados

Arquitectura de computadores – Semana 2, 3 y 4

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén



Riesgos de Control



Situaciones peligrosas (Riesgos)

- Un riesgo "hazard" es una situación en la que una instrucción planificada no puede ejecutarse en el ciclo de reloj "correcto".
 - Riesgo estructural: El hardware no admite el acceso al mismo recurso a través de múltiples instrucciones en el mismo ciclo.
 - Riesgo de datos: Las instrucciones tienen dependencia de datos.
 - Para ello, es necesario esperar a que la instrucción anterior complete su lectura/escritura de datos.
 - **Riesgos de control.** El flujo de procesamiento de las instrucciones depende de la instrucción anterior.
 - Normalmente este tipo de riesgo están asociados a las bifurcaciones (saltos y llamadas/retornos a rutinas)

Riesgos de control

- Situaciones donde el **flujo de instrucciones se interrumpe** debido a las instrucciones de salto (branch) en el programa, es decir, se produce una situación que modifica el contador del programa a un valor que no es $PC+4$.
- Se producen cuando las instrucciones de salto, ya sean **condicionales o incondicionales**, influyen en el orden en que se ejecutan las instrucciones en el pipeline
- Se **ralentizan la ejecución del programa** porque pueden causar que el pipeline deba vaciarse o reorganizarse debido a que las instrucciones incorrectas se han avanzado en el pipeline antes de que se hiciera efectivo el salto.
- Un **salto se hace efectivo** cuando se comienza a procesar la instrucción de dirección efectiva que ha modificado el contador del programa
- Para mitigar estos riesgos y minimizar los retrasos los procesadores modernos utilizan técnicas como la predicción de saltos y el desenrollado de bucles junto con el reordenamiento de código

Riesgos de control

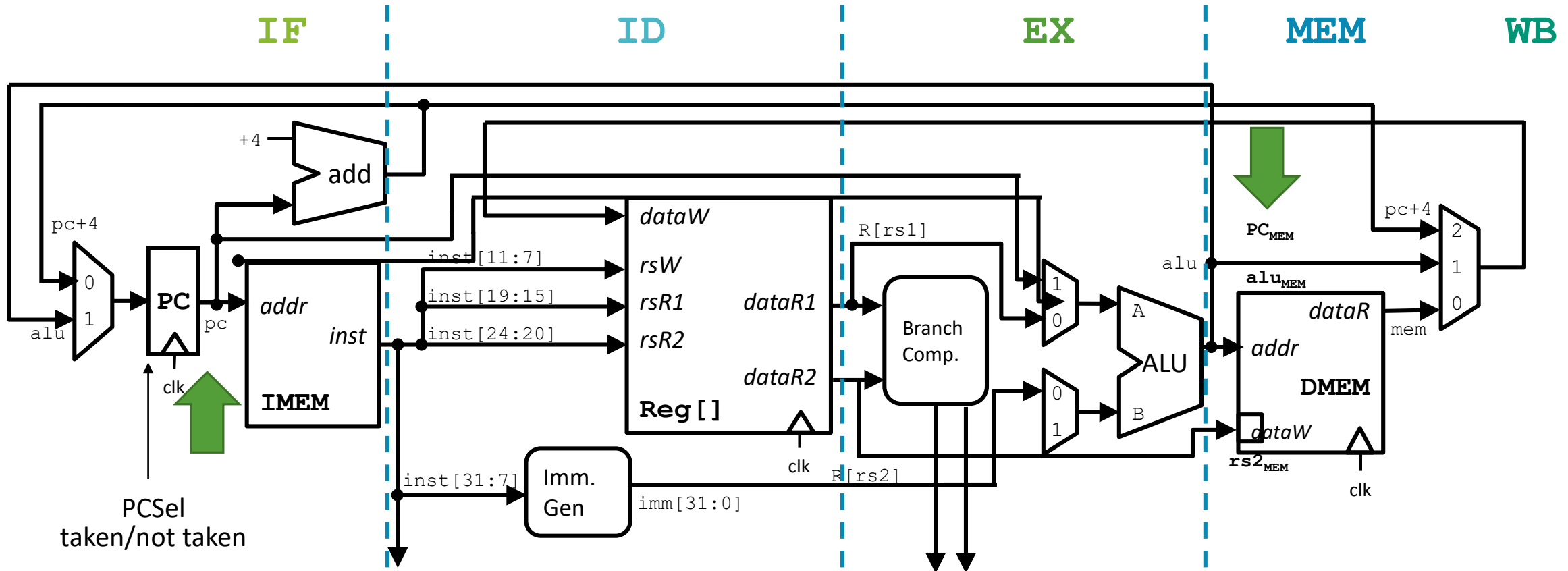
Instrucciones de Salto Incondicional

- Siempre hacen que el programa se desplace a una dirección específica (salto efectivo), independientemente de las condiciones.
 - Ejemplos de instrucciones de salto incondicional son las instrucciones de salto (jump) y las instrucciones de llamada y retorno (call y return).

Instrucciones de Salto Condicionales:

- Estas son instrucciones que dependen de una condición para decidir si se debe realizar un salto a otra dirección o no, es decir, el salto puede ser efectivo o no.
- Los riesgos de control son **especialmente problemáticos en las instrucciones de salto condicional** porque la decisión sobre qué dirección tomar (si el salto es efectivo o no) se toma en etapas posteriores del pipeline.

Resultado del salto calculado durante la MEM



- En la etapa MEM: EX/MEM el cauce alimenta al MUX de la etapa IF. Se activa el control PC. En el siguiente ciclo de reloj en la etapa IF, el PC se actualiza y se obtiene la instrucción correcta.

Riesgos de Control: Saltos Condicionales

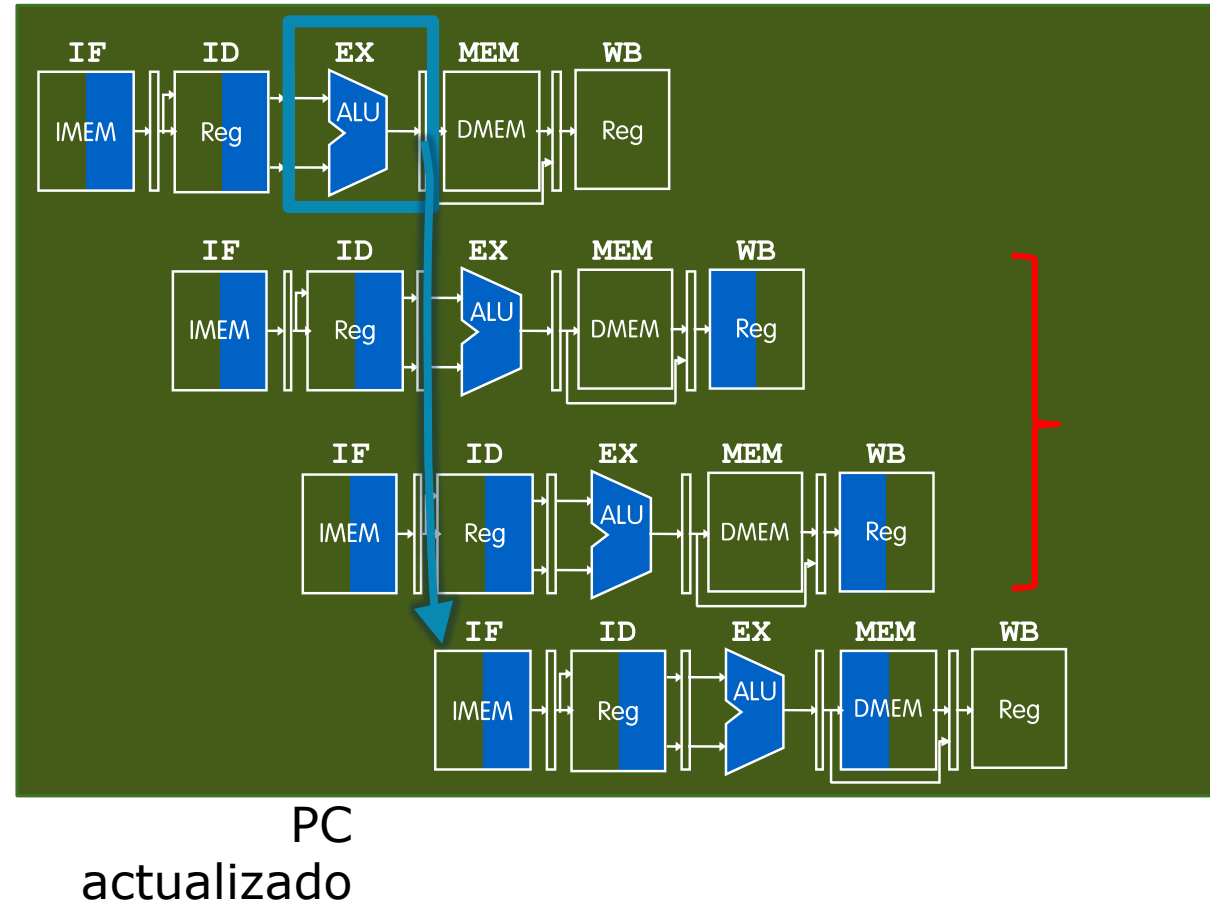
Los riesgos de control se producen cuando la instrucción obtenida puede no ser la necesaria. Por ejemplo, si se toma el salto en una instrucción beq:

0x40 beq t0,t1,Label

0x44 sub t2,s0,t0

0x48 or t6,s0,t3

0x70 sw s0,8(t3)



La ejecución de la instrucción comienza antes de que se conozca el resultado del salto.

La instrucción correcta comienza a ejecutarse

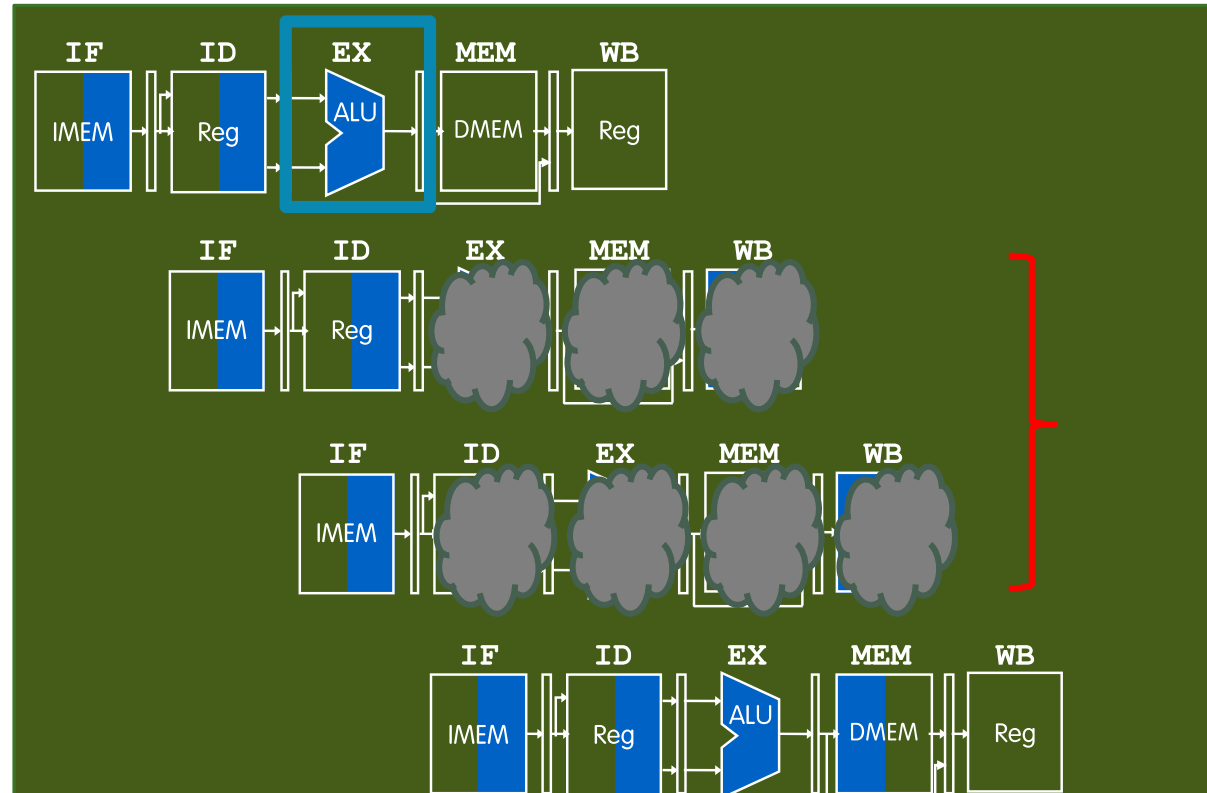
Abortar la ins. después de la bifurcación (Si tomado)

0x40 beq t0,t1,Label

0x44 sub t2,s0,t0

0x48 or t6,s0,t3

0x70 sw s0,8(t3)



Vaciar el cauce convirtiendo las instrucciones incorrectas en abortadas.

PC actualizado, instrucción correcta cargada

Predicción de saltos reducir las ciclos perdidos

- Cada salto/bifurcación tomado en el cauce RV32IM consume 3 ciclos de reloj.
 - Considerando que si no se toma en el salto, el cauce no se detiene y las instrucciones correctas se obtienen secuencialmente después de la instrucción de salto.
- Podemos **mejorar el rendimiento medio** de la CPU mediante la predicción del salto
 - Al principio del cauce, predecir en qué dirección irán los saltos y vaciar el cauce, es decir, abortar las instrucciones incorrectamente insertadas si la predicción del salto es incorrecta.

Veremos más técnicas de predicción de saltos en la última sección del tema.

Predictor Naïve. Salto no tomado (Don't Take Branch)

"Predecir" que el próximo PC será PC + 4

"Evaluar" la predicción

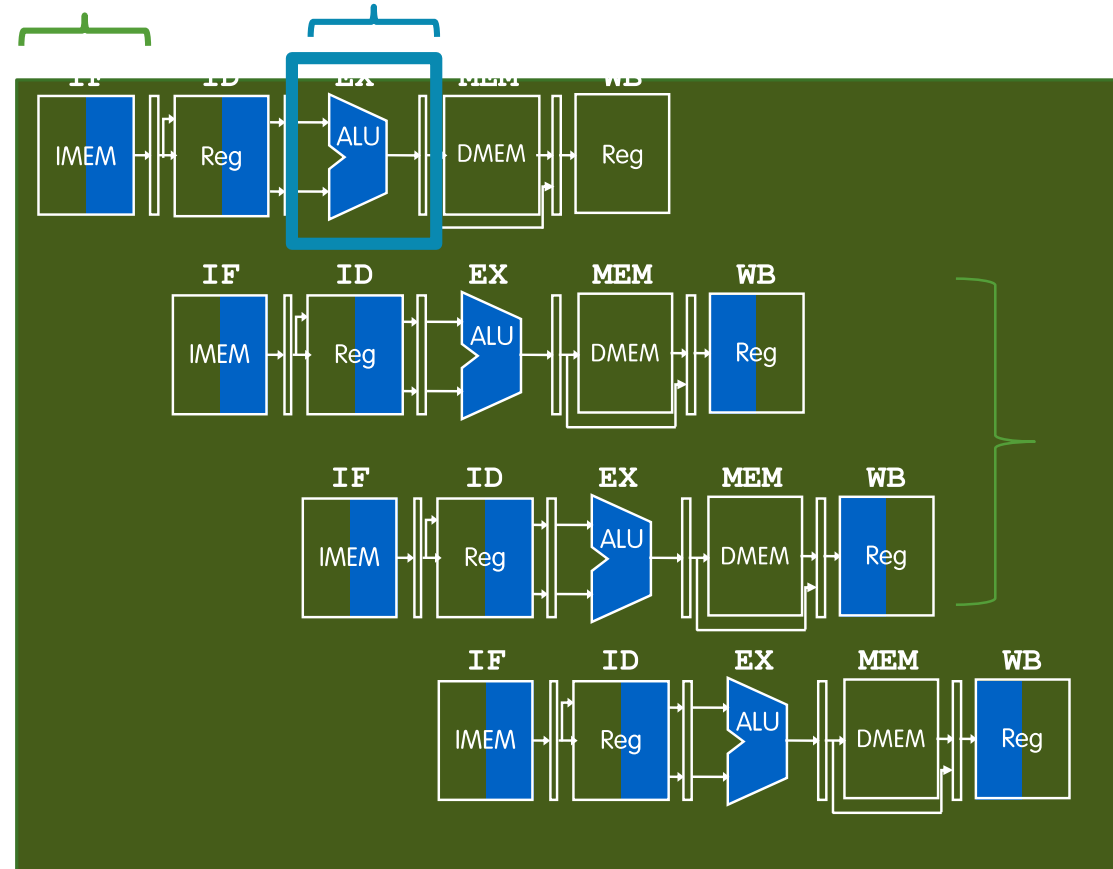
RV32IM "predice" **siempre que no se tomará el salto** (not taken).

0x40 beq t0,t1,Label

0x44 sub t2,s0,t0

0x48 or t6,s0,t3

0x50 add t2,s0,s0



Si no se toma el salto, ya se han procesado las instrucciones correctas.

La penalización por predicción incorrecta sigue siendo de 2 ciclos de reloj.
La predicción de saltos intenta mejorar el rendimiento medio.

Desenrollado de bucles

- El **desenrollado de bucles** es una técnica de optimización que mejora la eficiencia y el rendimiento de los programas al trabajar con estructuras repetitivas (bucles).
- El objetivo del desenrollado de bucles es:
 - Reducir la cantidad de operaciones repetitivas
 - Minimizar las instrucciones redundantes
 - Aprovechar al máximo los recursos de hardware disponibles
- El desarrollo de bucles implica la reestructuración del código fuente original para eliminar ineficiencias y mejorar la velocidad y el rendimiento.

Desenrollado de bucles. Reducir número de saltos

Ejemplo de código en C

```
for(i=0; i<1000; i++)  
x[i] = x[i] + s;
```

Ejemplo ensamblador

```
Loop:  lw    t0, 0(s0)  
       add   t0,t0,s1 # add s to array element  
       sw    t0,0(s0) # store result  
       addi  s0,s0,4  # move to next element  
       bne   s0,s2,Loop # repeat Loop if not done
```

Consideraciones:

- s0 → dirección inicial (parte superior del array)
- s1 → valor escalar s
- s2 → dirección de terminación (final del array)

Desenrollado de bucles. Reducir número de saltos

Consideraciones importantes:

- Este desenrollado funciona si el tamaño del vector es divisor de 4.
 - Para un vector de 1200 elementos, el bucle se ejecutará 300 veces.

Para 3 iteraciones dentro → 400 veces
Para 2 iteraciones dentro → 600 veces

Loop:

```
lw  t0,0(s0)
add t0,t0,s1
sw  t0,0(s0)
lw  t1,4(s0)
add t1,t1,s1
sw  t1,4(s0)
lw  t2,8(s0)
add t2,t2,s1
sw  t2,8(s0)
lw  t3,12(s0)
add t3,t3,s1
sw  t3,12(s0)
addi s0,s0,16
bne s0,s2,Loop
```

Desenrollado de bucles. Reducir número de saltos

Consideraciones importantes:

- Este desenrollado funciona si el tamaño del vector es divisor de 4.
 - Tengo que utilizar los desplazamientos en las cargas para moverme dentro de las posiciones.
 - Para el primer dato: `lw t0,0(s0)`
 - Para el segundo dato: `lw t1,4(s0)`
 - Para el tercer dato: `lw t2,8(s0)`
 - Para el cuarto dato: `lw t3,12(s0)`

Es muy importante el **uso de diferentes registros (t0, t1, t2 y t3)** para no generar detenciones por dependencias de datos.

El puntero s0 no se actualiza para cada lectura, se utilizan los desplazamientos en la carga

```
Loop:  lw t0,0(s0)
      add t0,t0,s1
      sw t0,0(s0)
      lw t1,4(s0)
      add t1,t1,s1
      sw t1,4(s0)
      lw t2,8(s0)
      add t2,t2,s1
      sw t2,8(s0)
      lw t3,12(s0)
      add t3,t3,s1
      sw t3,12(s0)
      addi s0,s0,16
      bne s0,s2,Loop
```

Desenrollado de bucles. Reducir número de saltos

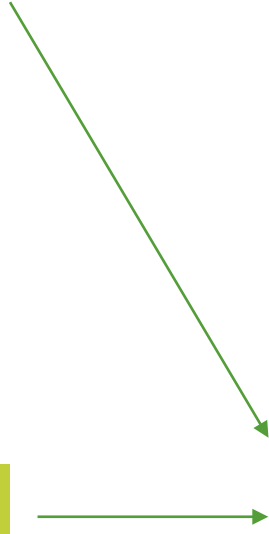
Consideraciones importantes:

Instrucción **addi** para actualizar el puntero, **s0**, solo se realiza una vez. Como en este caso se realizan 4 operaciones, tiene que desplazarse 4 posiciones que corresponden a 16 bytes.

Para 3 iteraciones → 12 bytes de desplazamiento
Para 2 iteraciones → 8 bytes de desplazamiento

Instrucción **bne** sólo se encuentra una vez dentro del bucle

Loop: `lw t0,0(s0)`
`add t0,t0,s1`
`sw t0,0(s0)`
`lw t1,4(s0)`
`add t1,t1,s1`
`sw t1,4(s0)`
`lw t2,8(s0)`
`add t2,t2,s1`
`sw t2,8(s0)`
`lw t3,12(s0)`
`add t3,t3,s1`
`sw t3,12(s0)`
`addi s0,s0,16`
`bne s0,s2,Loop`



Atención en las detenciones por carga de datos

Si después de una carga se utiliza una instrucción que utiliza como operando fuente el registro destino de la carga, se produce una detención.

Tenemos 4 situaciones con los pares de instrucciones `lw` y `add`.

Por cada vuelta del bucle se perderían 4 ciclos por detenciones por riesgos de datos en la carga.

```
Loop: { lw t0, 0(s0) }
      { add t0, t0, s1 }
      { sw t0, 0(s0) }

      { lw t1, 4(s0) }
      { add t1, t1, s1 }
      { sw t1, 4(s0) }

      { lw t2, 8(s0) }
      { add t2, t2, s1 }
      { sw t2, 8(s0) }

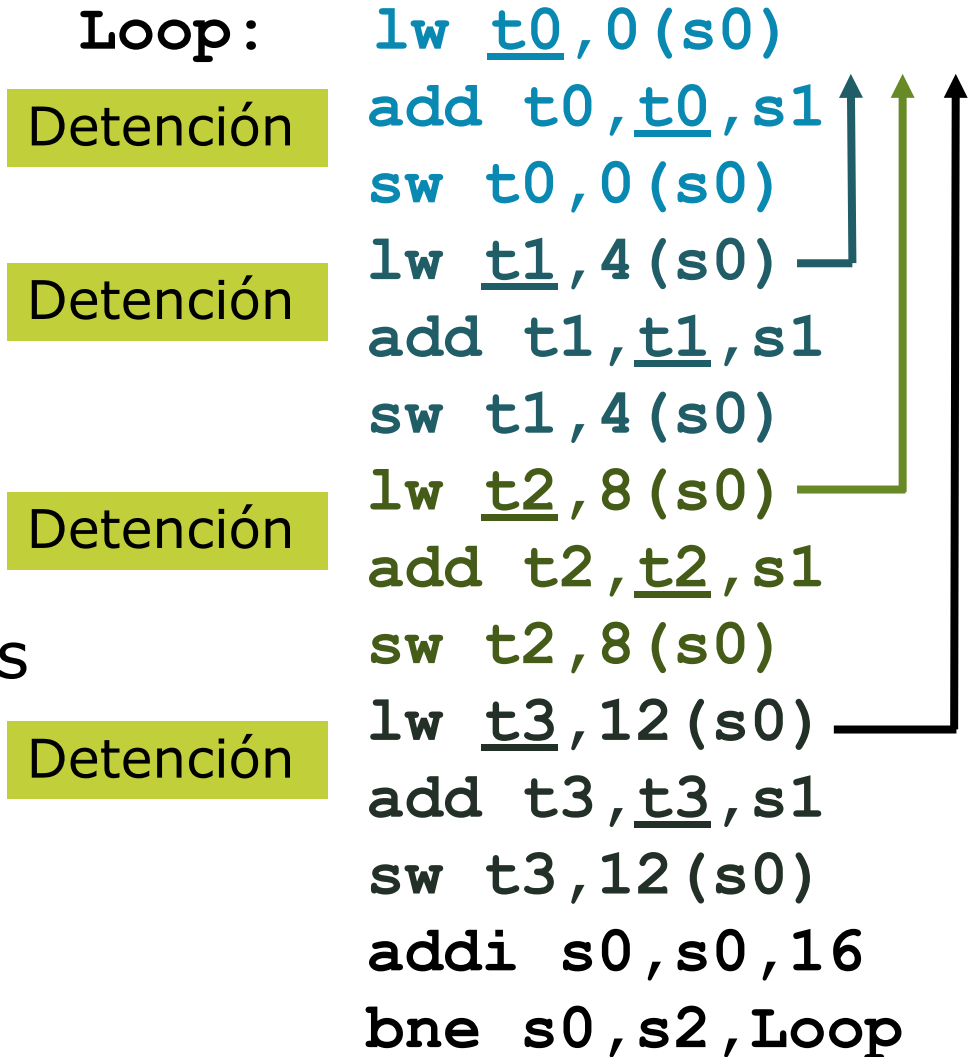
      { lw t3, 12(s0) }
      { add t3, t3, s1 }
      { sw t3, 12(s0) }
      { addi s0, s0, 16 }
      { bne s0, s2, Loop }
```


Atención en las detenciones por carga de datos

Si después de una carga se utiliza una instrucción que utiliza como operando fuente el registro destino de la carga, se produce una detención.

Tenemos 4 situaciones con los pares de instrucciones `lw` y `add`.

Por cada vuelta del bucle se perderían 4 ciclos por detenciones por riesgos de datos en la carga.



Desenrollado con reordenamiento de código

La **solución** es la misma que se aplicaba anteriormente: el **reordenamiento** de código.

Aplicando las dos técnicas podemos obtener el desenrollado de bucles reordenado que elimina las detenciones por riesgos de datos

Moviendo las instrucciones hemos eliminado las detenciones y evitamos la penalización de 4 ciclos perdidos.

```
Loop:  lw  t0,0(s0)
      lw  t1,4(s0)
      lw  t2,8(s0)
      lw  t3,12(s0)
      add t0,t0,s1
      sw  t0,0(s0)
      add t1,t1,s1
      sw  t1,4(s0)
      add t2,t2,s1
      sw  t2,8(s0)
      add t3,t3,s1
      sw  t3,12(s0)
      addi s0,s0,16
      bne s0,s2,Loop
```

Predicción de saltos



Ejecución especulativa

- La ejecución especulativa es una estrategia que se usa en la mayoría de procesadores de altas prestaciones.
- Consiste en realizar la ejecución de las instrucciones o partes de instrucciones, antes de estar seguro de si la ejecución se requiere.
- Los procesadores usan ejecución especulativa para reducir el coste de las instrucciones de salto condicional no resueltos.
 - Así, cuando se encuentra un salto condicional, el procesador realiza una predicción sobre cual es el camino más probable a seguir (utilizando técnicas de predicción de saltos), e inmediatamente prosigue la captación, decodificación y ejecución de instrucciones desde dicho punto, sin esperar a saber si es el camino correcto.
- Si, posteriormente, la predicción resulta ser errónea, el procesador descarta las instrucciones ejecutadas a partir del punto de salto, y continúa la ejecución de las instrucciones del camino correcto. Si la predicción resulta correcta, el procesador 'retira' las instrucciones ejecutadas y continua la ejecución de las instrucciones.

Ejecución especulativa

- La ejecución especulativa evita la introducción de ciclos de retardo, aumentando la eficiencia de la ejecución y las prestaciones de la máquina a costa de utilizar al máximo sus unidades funcionales.
- Sin embargo, los ciclos utilizados en las ejecuciones especulativas consumen ciclos de CPU y por tanto un mayor consumo. Así, se ejecutan muchas más instrucciones de las que necesita el flujo del programa.
- Es necesario un mecanismo con el que deshacer la ejecución de una instrucción especulativa, cuando se comprueba que ésta no debería de ser procesada si hubiera sido en una máquina secuencial.
- Este mecanismo consiste en que:
 - El almacenamiento y los registros visibles no se pueden actualizar inmediatamente después de su ejecución.
 - Se han de mantener en algún tipo de almacenamiento temporal para después convertirlo en permanente una vez que se determine que el modelo secuencial habría ejecutado la instrucción.

Ejecución especulativa

- En estas condiciones es posible descartar una instrucción ejecutada simplemente ignorando el valor almacenado temporalmente y liberando el almacenamiento y registros bloqueados.
- Por otro lado, si se verifica que la instrucción ejecutada corresponde al camino correcto, el proceso de retiro de la instrucción simplemente refresca el registro bloqueado con el valor contenido en el almacenamiento temporal.
- La utilización de la ejecución especulativa y la predicción de saltos implica que algunas instrucciones pueden completar su ejecución y después ser desechadas porque la bifurcación que llevaba a ellas no se produjo.

Recursos

Descripción

Para ampliar tus conocimientos sobre los contenidos de esta semana te recomendamos que consultes los recursos indicados a continuación.

Recursos

- Arquitectura de Computadores de Julio Ortega, Alberto Prieto y Mancia Anguita. Ediciones Paraninfo – 978849732274
- Great Ideas in Computer Architecture (Machine Structures) CS 61C at UC Berkeley with Dan Garcia, Justin Yokota - Spring 2025, Lección 21, 22 y 23([link](#))
- *Nota: Se reconoce los derechos de parte del material a Dan Garcia y a Justin Yokota en CS 61C at UC Berkeley with Spring 2023*